Resource Builder — Project Overview

Sep 28, 2026 · @Pratham

What we're building

A tool where an organisation designs a fundraising resource once, and every fundraiser automatically gets their own personalised version. Visitors can see it on a web page and download it.

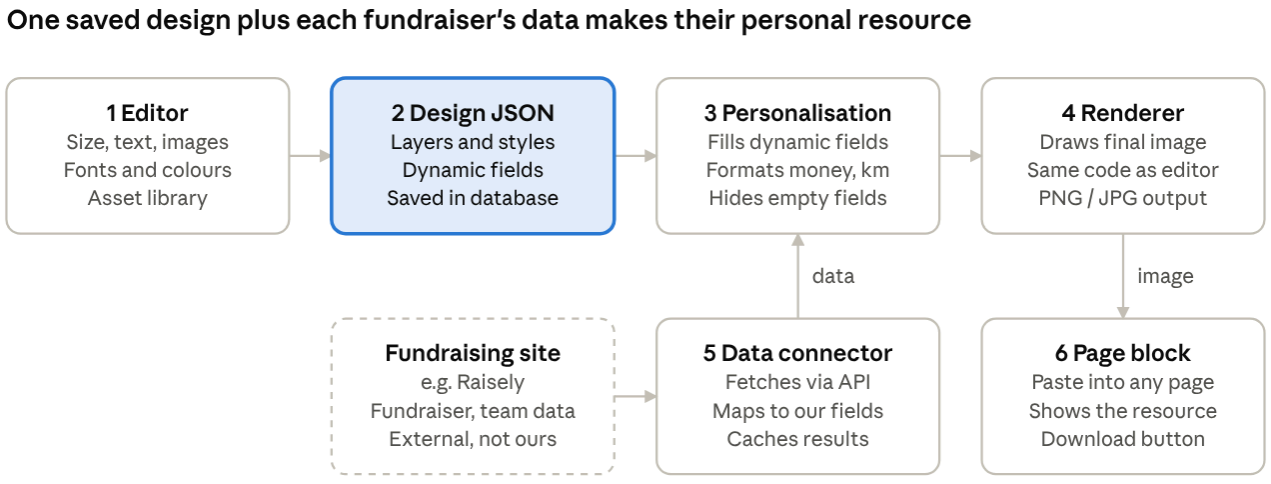
A user goes through four steps:

1. **Create** — design the resource in the editor: size, text, images, fonts, colours, assets.
2. **Personalise** — mark parts as dynamic, such as the fundraiser's name, photo, goal or amount raised.
3. **Connect** — the tool fetches each fundraiser's data from the fundraising platform and fills those parts in.
4. **Publish** — paste a block into any page; it shows the personalised resource with a Download button.

Example: one "I'm running for Sarah" template becomes Peter's running post, Alex's walking post and Alan's cycling post, with no redesign.

Core architecture

The design and the data travel separately and only meet at personalisation, just before the image is drawn.



core architecture · 6 parts

Read it left to right: the editor saves a design as JSON, personalisation fills in one fundraiser's data, and the renderer draws the image the page block shows. The bottom row brings data in from the fundraising site and sends the finished resource out to the page.

Project components

Six parts, numbered as in the diagram, built in order over 9 weeks (29 Sep – 27 Nov 2026).

#	Component	What it does	Built in
1	Editor	Create a resource; set and change size; add text, images and shapes; choose fonts and colours; move, resize, rotate; upload and reuse assets; undo/redo	Weeks 2–4
2	Design JSON	Saves every design as structured data (canvas, layers, styles, dynamic fields) instead of a flat image, so it can be reopened and reused	Weeks 1–4
3	Personalisation	Swaps dynamic fields for real values, formats money, % and distance, hides a field when data is missing	Week 5
4	Renderer	Turns the design plus the data into the final PNG/JPG, using the same drawing code as the editor	Weeks 5–7
5	Data connector	Logs in to the fundraising platform's API, fetches fundraiser and team data, maps it to our own field names	Week 6
6	Page block	A snippet pasted into any page that shows the personalised resource with a Download button	Week 7

Weeks 8–9 are testing, fixes and buffer.

Technology

A JavaScript/TypeScript stack from end to end, so one person can work across the whole product in one language.

Layer	Technology	Why
Editor (frontend)	React + TypeScript	Standard, well-supported UI framework; types catch mistakes early
Canvas	Konva.js (react-konva)	Ready-made drag, resize, rotate, layers and export to image
Fonts	Google Fonts + uploaded font files	Large free font library, plus the organisation's brand fonts
Backend / API	Node.js (NestJS or Express)	Same language as the frontend; simple REST API
Database	PostgreSQL (JSONB)	Stores Design JSON as-is while keeping users and assets relational
File storage	Local disk now, S3-compatible later	Holds uploaded images and fonts; swappable at hosting time
Rendering	Konva on the server (node-canvas)	Same drawing code as the editor, so downloads match the preview
Fundraising data	Platform REST API (e.g. Raisely)	Fetches fundraiser and team data by API key
Page block	Small JavaScript embed + image URL fallback	Works on pages that allow scripts and pages that don't

Hosting and deployment are not part of this phase.

Key principles

The main one: **design once, personalise for everyone.** Four rules make that work.

1. **Keep the four parts separate.** The editor, the saved design, the data connector and the page block each do one job. Changing one never breaks the others.
2. **Save designs as JSON, never as flat images.** JSON can be reopened, edited, reused and filled with anyone's data. A flat image can't.
3. **Use our own field names.** Templates say `fundraiser.name`, not the platform's field name. Adding a second fundraising platform means writing one new connector, with no changes to the editor or any template.
4. **One drawing engine.** The editor preview, the page block and the download all use the same rendering code, so what you design is exactly what people download.

Explaining it in one minute

A script to say out loud, one line per point:

1. **Problem:** charities want every fundraiser to share a personal graphic, but designing one per person doesn't scale.
2. **Solution:** design a resource once; the tool fills in each fundraiser's name, photo, goal and progress automatically.
3. **How:** an editor saves the design as JSON, a connector pulls fundraiser data from the fundraising platform, and a renderer draws each person's version.
4. **Delivery:** paste one block into a page; visitors see the personalised resource and can download it.
5. **Why it lasts:** the parts are separate, so adding another fundraising platform or output format later doesn't mean rebuilding.
6. **Plan:** I'm building it solo in 9 weeks, 29 Sep to 27 Nov, with a demo every Friday and an editor demo on 23 Oct.